

THE RELATIONAL DATA MODEL

The relational data model represents data in the form of tables. The relational data model consists of the following three components (Fleming and von Halle, 1989):

1. *Data structure* Data are organized in the form of tables, with rows and columns.
2. *Data manipulation* Powerful operations (using the SQL language) are used to manipulate data stored in the relations.
3. *Data integrity* the model includes mechanisms to specify business rules that maintain the integrity of data when they are manipulated.

A relation is a named, two-dimensional table of data. Each relation (or table) consists of a set of named columns and an arbitrary number of unnamed rows.

RELATIONAL KEYS We must be able to store and retrieve a row of data in a relation, based on the data values stored in that row. To achieve this goal, every relation must have a primary key.

A primary key is an attribute or a combination of attributes that uniquely identifies each row in a relation. We designate a primary key by underlining the attribute name(s). For example, the primary key for the relation EMPLOYEE1 is EmpID.

A composite key is a primary key that consists of more than one attribute. For example, the primary key for a relation DEPENDENT would likely consist of the combination EmpID and DependentName.

A foreign key is an attribute (possibly composite) in a relation that serves as the primary key of another relation. For example, consider the relations EMPLOYEE1 and DEPARTMENT.

PROPERTIES OF RELATIONS We have defined relations as two-dimensional tables of data. However, not all tables are relations. Relations have several properties that distinguish them from non-relational tables. We summarize these properties next:

1. Each relation (or table) in a database has a unique name.
2. An entry at the intersection of each row and column is atomic (or single valued). There can be only one value associated with each attribute on a specific row of a table; no multivalued attributes are allowed in a relation.
3. Each row is unique; no two rows in a relation can be identical.
4. Each attribute (or column) within a table has a unique name.
5. The sequence of columns (left to right) is insignificant. The order of the columns in a relation can be changed without changing the meaning or use of the relation.
6. The sequence of rows (top to bottom) is insignificant. As with columns, the order of the rows of a relation may be changed or stored in any sequence.

Transaction Management and concurrency control

Database transactions reflect real-world transactions that are triggered by events such as buying a product, registering for a course, or making a deposit in a checking account. Transactions are likely to contain many parts. For example, a sales transaction might require updating the customer's account, adjusting the product inventory, and updating the seller's accounts receivable. All parts of a transaction must be successfully completed to prevent data integrity problems.

The main database transaction properties are atomicity, consistency, isolation, and durability. In addition, serializability is a characteristic of the schedule of operations for the execution of concurrent transactions.

To understand the concept of a transaction, suppose that you sell a product to a customer. Furthermore, suppose that the customer may charge the purchase to his or her account. Given that scenario, your sales transaction consists of at least the following parts:

- You must write a new customer invoice.
- You must reduce the quantity on hand in the product's inventory.
- You must update the account transactions.

- You must update the customer balance.

A transaction may consist of a simple `SELECT` statement to generate a list of table contents; it may consist of a series of related `UPDATE` statements to change the values of attributes in various tables; it may consist of a series of `INSERT` statements to add rows to one or more tables, or it may consist of a combination of `SELECT`, `UPDATE`, and `INSERT` statements. The sales transaction example includes a combination of `INSERT` and `UPDATE` statements.

A transaction is a *logical* unit of work that must be entirely completed or entirely aborted; no intermediate states are acceptable. In other words, a multicomponent transaction, such as the previously mentioned sale, must not be partially completed. Updating only the inventory or only the accounts receivable is not acceptable. All of the SQL statements in the transaction must be completed successfully. If any of the SQL statements fail, the entire transaction is rolled back to the original database state that existed before the transaction started. A successful transaction changes the database from one consistent state to another. A consistent database state is one in which all data integrity constraints are satisfied.

Transaction Properties

Each individual transaction must display *atomicity*, *consistency*, *isolation*, and *durability*. These properties are sometimes referred to as the ACID test. In addition, when executing multiple transactions, the DBMS must schedule the concurrent execution of the transaction's operations. The schedule of such transaction's operations must exhibit the property of *serializability*.

- Atomicity requires that *all* operations (SQL requests) of a transaction be completed; if not, the transaction is aborted. If a transaction T1 has four SQL requests, all four requests must be successfully completed; otherwise, the entire transaction is aborted. In other words, a transaction is treated as a single, indivisible, logical unit of work.
- Consistency indicates the permanence of the database's consistent state. A transaction takes a database from one consistent state to another consistent state. When a transaction is completed, the database must be in a consistent state; if any of the transaction parts violates an integrity constraint, the entire transaction is aborted.
- Isolation means that the data used during the execution of a transaction cannot be used by a second transaction until the first one is completed. In other words, if a transaction T1 is being executed and is using the data item X, that data item cannot be accessed by any other transaction (T2 ... Tn) until T1 ends. This property is particularly useful in multiuser database environments because several users can access and update the database at the same time.
- Durability ensures that once transaction changes are done (committed), they cannot be undone or lost, even in the event of a system failure.
- Serializability ensures that the schedule for the concurrent execution of the transactions yields consistent results. This property is important in multiuser and distributed databases, where multiple transactions are likely to be executed concurrently. Naturally, if only a single transaction is executed, serializability is not an issue.

Transaction Management with SQL

The American National Standards Institute (ANSI) has defined standards that govern SQL database transactions.

Transaction support is provided by two SQL statements: `COMMIT` and `ROLLBACK`. The ANSI standards require that when a transaction sequence is initiated by a user or an application program, the sequence must continue through all succeeding SQL statements until one of the following four events occurs:

- A `COMMIT` statement is reached, in which case all changes are permanently recorded within the database. The `COMMIT` statement automatically ends the SQL transaction.

- A ROLLBACK statement is reached, in which case all changes are aborted and the database is rolled back to its previous consistent state.
- The end of a program is successfully reached, in which case all changes are permanently recorded within the database. This action is equivalent to COMMIT.
- The program is abnormally terminated, in which case the changes made in the database are aborted and the database is rolled back to its previous consistent state. This action is equivalent to ROLLBACK.

The use of COMMIT is illustrated in the following simplified sales example, which updates a product's quantity on hand (PROD_QOH) and the customer's balance when the customer buys two units of product 1558-QW1 priced at \$43.99 per unit (for a total of \$87.98) and charges the purchase to the customer's account:

```
UPDATE PRODUCT
SET PROD_QOH = PROD_QOH - 2
WHERE PROD_CODE = '1558-QW1';
UPDATE CUSTOMER
SET CUST_BALANCE = CUST_BALANCE + 87.98
WHERE CUST_NUMBER = '10011';
COMMIT;
```

Actually, the COMMIT statement used in that example is not necessary if the UPDATE statement is the application's last action and the application terminates normally. However, good programming practice dictates that you include the COMMIT statement at the end of a transaction declaration.

A transaction begins implicitly when the first SQL statement is encountered. Not all SQL implementations follow the

ANSI standard; some (such as SQL Server) use transaction management statements such as:

```
BEGIN TRANSACTION;
```

To indicate the beginning of a new transaction.

The Transaction Log

A DBMS uses a transaction log to keep track of all transactions that update the database. The information stored in this log is used by the DBMS for a recovery requirement triggered by a ROLLBACK statement, a program's abnormal termination, or a system failure such as a network discrepancy or a disk crash. Some RDBMSs use the transaction log to recover a database *forward* to a currently consistent state. While the DBMS executes transactions that modify the database, it also automatically updates the transaction log. The transaction log stores:

- A record for the beginning of the transaction.
- For each transaction component (SQL statement):
 - The type of operation being performed (update, delete, insert).
 - The names of the objects affected by the transaction (the name of the table).
 - The "before" and "after" values for the fields being updated.
 - Pointers to the previous and next transaction log entries for the same transaction.
- The ending (COMMIT) of the transaction.

Although using a transaction log increases the processing overhead of a DBMS, the ability to restore a corrupted database is worth the price.

CONCURRENCY CONTROL

The coordination of the simultaneous execution of transactions in a multiuser database system is known as concurrency control. The objective of concurrency control is to ensure the serializability of transactions in a multiuser database environment. Concurrency control is important because the simultaneous

execution of transactions over a shared database can create several data integrity and consistency problems. The three main problems are lost updates, uncommitted data, and inconsistent retrievals.

Lost Updates

The lost update problem occurs when two concurrent transactions, T1 and T2, are updating the same data element and one of the updates is lost (overwritten by the other transaction). To see an illustration of lost updates, let's examine a simple PRODUCT table. One of the PRODUCT table's attributes is a product's quantity on hand (PROD_QOH). Assume that you have a product whose current PROD_QOH value is 35. Also assume that two concurrent transactions, T1 and T2, occur that update the PROD_QOH value for some item in the PRODUCT table. The transactions are shown below.

TRANSACTION	COMPUTATION
T1: Purchase 100 units	$\text{PROD_QOH} = \text{PROD_QOH} + 100$
T2: Sell 30 units	$\text{PROD_QOH} = \text{PROD_QOH} - 30$

But suppose that a transaction is able to read a product's PROD_QOH value from the table *before* a previous transaction (using the *same* product) has been committed. The sequence depicted in the Table below shows how the lost update problem can arise. Note that the first transaction (T1) has not yet been committed when the second transaction (T2) is executed. Therefore, T2 still operates on the value 35, and its subtraction yields 5 in memory. In the meantime, T1 writes the value 135 to disk, which is promptly overwritten by T2. In short, the addition of 100 units is "lost" during the process.

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T2	Read PROD_QOH	35
3	T1	$\text{PROD_QOH} = 35 + 100$	
4	T2	$\text{PROD_QOH} = 35 - 30$	
5	T1	Write PROD_QOH (Lost update)	135
6	T2	Write PROD_QOH	5

Uncommitted Data

The phenomenon of uncommitted data occurs when two transactions, T1 and T2, are executed concurrently and the first transaction (T1) is rolled back after the second transaction (T2) has already accessed the uncommitted data—thus violating the isolation property of transactions. To illustrate that possibility, let's use the same transactions described during the lost updates discussion. T1 has two atomic parts to it, one of which is the update of the inventory, the other possibly being the update of the invoice total (not shown). T1 is forced to roll back due to an error during the updating of the invoice's total; hence, it rolls back all the way, undoing the inventory update as well. This time the T1 transaction is rolled back to eliminate the addition of the 100 units. (See Table below) Because T2 subtracts 30 from the original 35 units, the correct answer should be 5.

TRANSACTION	COMPUTATION
T1: Purchase 100 units	$\text{PROD_QOH} = \text{PROD_QOH} + 100$ (Rolled back)
T2: Sell 30 units	$\text{PROD_QOH} = \text{PROD_QOH} - 30$

The table below shows how, under normal circumstances, the serial execution of those transactions yields the correct answer.

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T1	PROD_QOH = 35 + 100	
3	T1	Write PROD_QOH	135
4	T1	*****ROLLBACK *****	35
5	T2	Read PROD_QOH	35
6	T2	PROD_QOH = 35 – 30	
7	T2	Write PROD_QOH	5

The table below shows how the uncommitted data problem can arise when the ROLLBACK is completed after T2 has begun its execution.

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T1	PROD_QOH = 35 + 100	
3	T1	Write PROD_QOH	135
4	T2	Read PROD_QOH (Read uncommitted data)	135
5	T2	PROD_QOH = 135 – 30	
6	T1	***** ROLLBACK *****	35
7	T2	Write PROD_QOH	105

Inconsistent Retrievals

Inconsistent retrievals occur when a transaction accesses data before and after another transaction(s) finish working with such data. For example, an inconsistent retrieval would occur if transaction T1 calculated some summary (aggregate) function over a set of data while another transaction (T2) was updating the same data. The problem is that the transaction might read some data before they are changed and other data *after* they are changed, thereby yielding inconsistent results.

To illustrate that problem, assume the following conditions:

1. T1 calculates the total quantity on hand of the products stored in the PRODUCT table.
2. At the same time, T2 updates the quantity on hand (PROD_QOH) for two of the PRODUCT table's products.

The two transactions are shown in the table 1 below:

TRANSACTION T1		TRANSACTION T2	
SELECT	SUM(PROD_QOH)	UPDATE	PRODUCT
FROM	PRODUCT	SET	PROD_QOH = PROD_QOH + 10
		WHERE	PROD_CODE = '1546-QQ2'
		UPDATE	PRODUCT
		SET	PROD_QOH = PROD_QOH – 10
		WHERE	PROD_CODE = '1558-QW1'
		COMMIT;	

While T1 calculates the total quantity on hand (PROD_QOH) for all items, T2 represents the correction of a typing error: the user added 10 units to product 1558-QW1's PROD_QOH but *meant* to add the 10 units to product

1546-QQ2's PROD_QOH. To correct the problem, the user adds 10 to product 1546-QQ2's PROD_QOH and subtracts 10 from product 1558-QW1's PROD_QOH. See table 2 below.

	BEFORE	AFTER
PROD_CODE	PROD_QOH	PROD_QOH
11QER/31	8	8
13-Q2/P2	32	32
1546-QQ2	15	$(15 + 10) \rightarrow 25$
1558-QW1	23	$(23 - 10) \rightarrow 13$
2232-QTY	8	8
2232-QWE	6	6
Total	92	92

Although the final results shown in Table 1 are correct after the adjustment, Table 3 demonstrates that inconsistent retrievals are possible during the transaction execution, making the result of T1's execution incorrect. The "After" summation shown in Table 2 reflects the fact that the value of 25 for product 1546-QQ2 was read *after* the WRITE statement was completed. Therefore, the "After" total is $40 + 25 = 65$. The "Before" total reflects the fact that the value of 23 for product 1558-QW1 was read *before* the next WRITE statement was completed to reflect the corrected update of 13. Therefore, the "Before" total is $65 + 23 = 88$.

TIME	TRANSACTION	ACTION	VALUE	TOTAL
1	T1	Read PROD_QOH for PROD_CODE = '11QER/31'	8	8
2	T1	Read PROD_QOH for PROD_CODE = '13-Q2/P2'	32	40
3	T2	Read PROD_QOH for PROD_CODE = '1546-QQ2'	15	
4	T2	PROD_QOH = $15 + 10$		
5	T2	Write PROD_QOH for PROD_CODE = '1546-QQ2'	25	
6	T1	Read PROD_QOH for PROD_CODE = '1546-QQ2'	25	(After) 65
7	T1	Read PROD_QOH for PROD_CODE = '1558-QW1'	23	(Before) 88
8	T2	Read PROD_QOH for PROD_CODE = '1558-QW1'	23	
9	T2	PROD_QOH = $23 - 10$		
10	T2	Write PROD_QOH for PROD_CODE = '1558-QW1'	13	
11	T2	***** COMMIT *****		
12	T1	Read PROD_QOH for PROD_CODE = '2232-QTY'	8	96
13	T1	Read PROD_QOH for PROD_CODE = '2232-QWE'	6	102

The computed answer of 102 is obviously wrong because you know from Table 2 that the correct answer is 92.

Unless the DBMS exercises concurrency control, a multiuser database environment can create havoc within the information system.

The Scheduler

The scheduler is a special DBMS process that establishes the order in which the operations within concurrent transactions are executed. The scheduler *interleaves* the execution of database operations to ensure serializability and isolation of transactions. To determine the appropriate order, the scheduler bases its actions on concurrency control algorithms, such as locking or time stamping methods.

The scheduler also makes sure that the computer's central processing unit (CPU) and storage systems are used efficiently. If there were no way to schedule the execution of transactions, all transactions would be executed on a first-come, first-served basis. The problem with that approach is that processing time is wasted when the CPU waits for a READ or WRITE operation to finish, thereby losing several CPU cycles.

Additionally, the scheduler facilitates data isolation to ensure that two transactions do not update the same data element at the same time. Database operations might require READ and/or WRITE actions that produce conflicts. For example, the Table below shows the possible conflict scenarios when two

transactions, T1 and T2, are executed concurrently over the same data. Note that in the Table below, two operations are in conflict when they access the same data and at least one of them is a WRITE operation.

	TRANSACTIONS		RESULT
	T1	T2	
Operations	Read	Read	No conflict
	Read	Write	Conflict
	Write	Read	Conflict
	Write	Write	Conflict

CONCURRENCY CONTROL WITH LOCKING METHODS

A lock guarantees exclusive use of a data item to a current transaction. In other words, transaction T2 does not have access to a data item that is currently being used by transaction T1. A transaction acquires a lock prior to data access; the lock is released (unlocked) when the transaction is completed so that another transaction can lock the data item for its exclusive use. This series of locking actions assumes that there is a likelihood of concurrent transactions attempting to manipulate the same data item at the same time. The use of locks based on the assumption that conflict between transactions is likely to occur is often referred to as pessimistic locking.

Lock Granularity

Lock granularity indicates the level of lock use. Locking can take place at the following levels: database, table, page, row, or even field (attribute).

Database Level

In a database-level lock, the entire database is locked, thus preventing the use of any tables in the database by transaction T2 while transaction T1 is being executed. This level of locking is good for batch processes, but it is unsuitable for multiuser DBMSs. You can imagine how s-l-o-w the data access would be if thousands of transactions had to wait for the previous transaction to be completed before the next one could reserve the entire database.

The Figure below illustrates the database-level lock. Note that because of the database-level lock, transactions T1 and T2 cannot access the same database concurrently *even when they use different tables*.

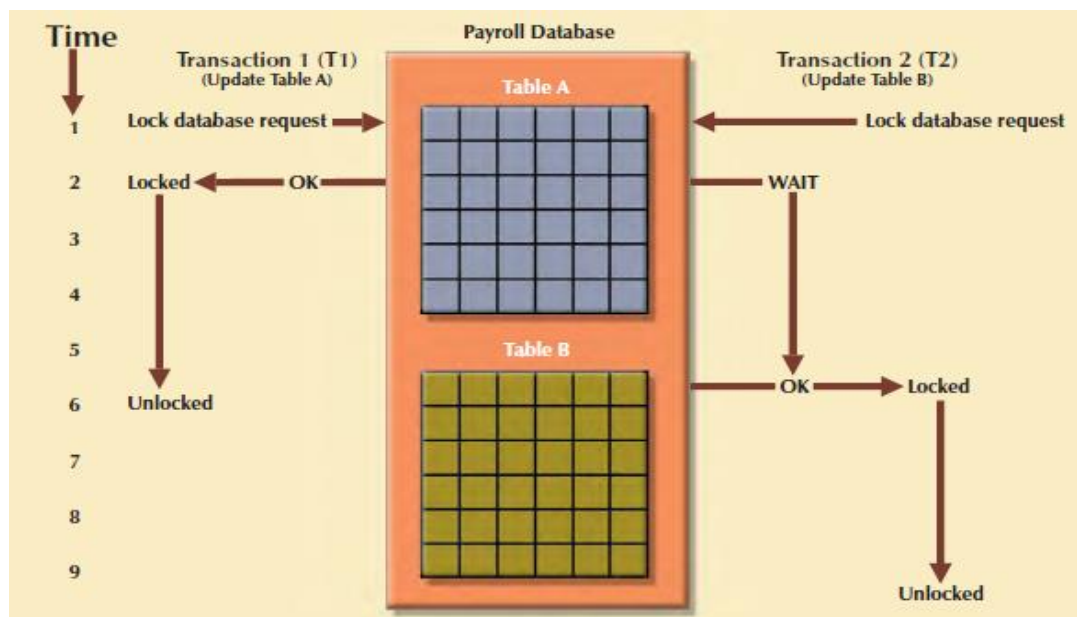
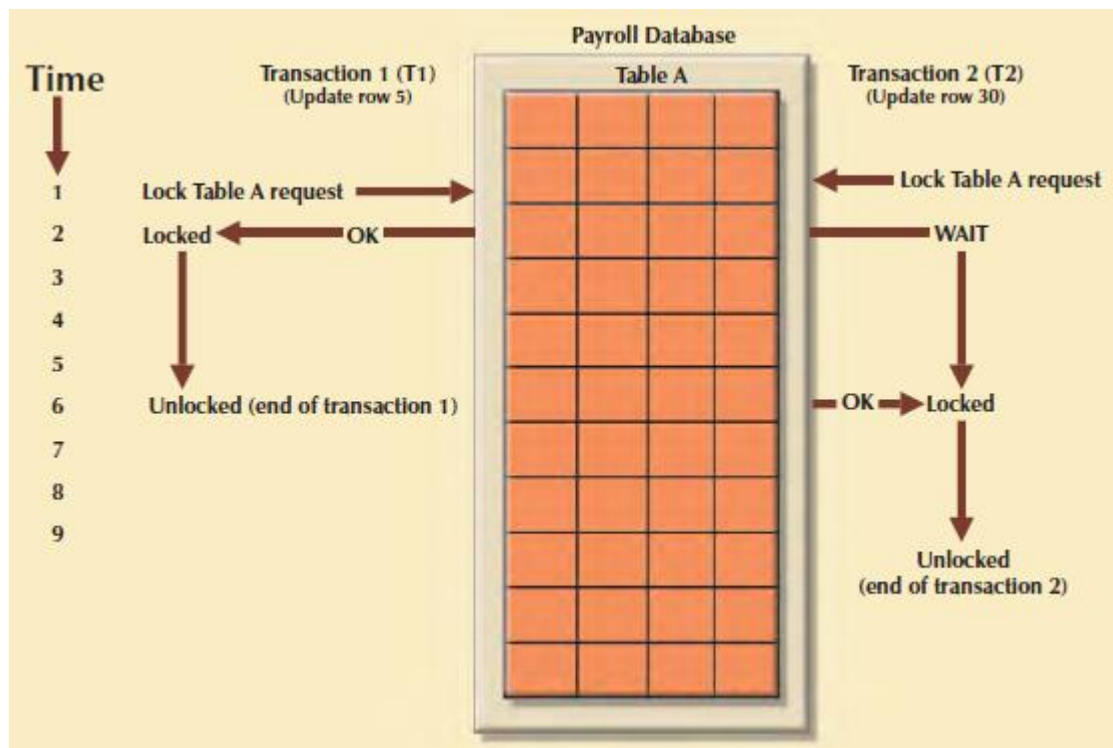


Table Level

In a table-level lock, the entire table is locked, preventing access to any row by transaction T2 while transaction T1 is using the table. If a transaction requires access to several tables, each table may be locked. However, two transactions can access the same database as long as they access different tables. Table-level locks, while less restrictive than database-level locks, can cause traffic jams when many transactions are waiting to access the same table.

Such a condition is especially irksome if the lock forces a delay when different transactions require access to different parts of the same table, that is, when the transactions would not interfere with each other. Consequently, table-level locks are not suitable for multiuser DBMSs

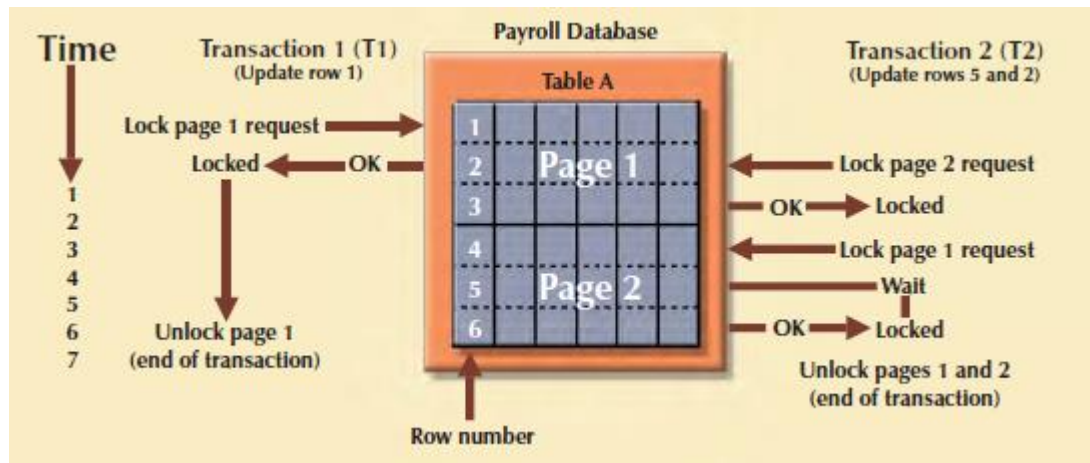


Page Level

In a page-level lock, the DBMS will lock an entire diskpage. A diskpage, or page, is the equivalent of a *diskblock*, which can be described as a directly addressable section of a disk. A page has a fixed size, such as 4K, 8K, or 16K.

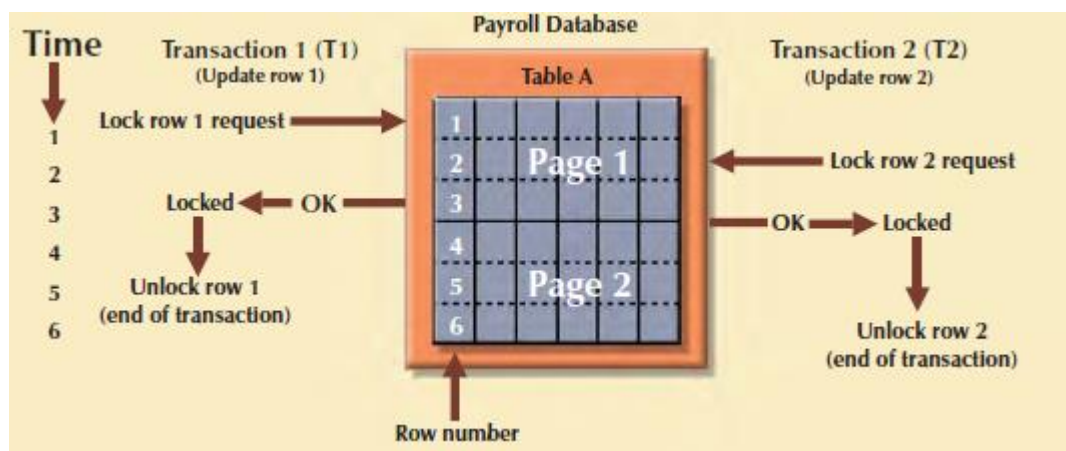
For example, if you want to write only 73 bytes to a 4K page, the entire 4K page must be read from disk, updated in memory, and written back to disk. A table can span several pages, and a page can contain several rows of one or more tables. Page-level locks are currently the most frequently used multiuser DBMS locking method.

If T2 requires the use of a row located on a page that is locked by T1, T2 must wait until the page is unlocked by T1.



Row Level

A row-level lock is much less restrictive than the locks discussed earlier. The DBMS allows concurrent transactions to access different rows of the same table even when the rows are located on the same page. Although the row-level locking approach improves the availability of data, its management requires high overhead because a lock exists for each row in a table of the database involved in a conflicting transaction. Modern DBMSs automatically escalate a lock from a row-level to a page-level lock when the application session requests multiple locks on the same page.



Note in the Figure above that both transactions can execute concurrently, even when the requested rows are on the same page. T2 must wait only if it requests the same row as T1.

Field Level

The field-level lock allows concurrent transactions to access the same row as long as they require the use of different fields (attributes) within that row. Although field-level locking clearly yields the most flexible multiuser data access, it is rarely implemented in a DBMS because it requires an extremely high level of computer overhead and because the row-level lock is much more useful in practice.

Lock Types

Regardless of the level of locking, the DBMS may use different lock types: binary or shared/exclusive.

Binary Locks

A binary lock has only two states: locked (1) or unlocked (0). If an object—that is, a database, table, page, or row—is locked by a transaction, no other transaction can use that object. If an object is unlocked, any transaction can lock the object for its use. Every database operation requires that the affected object be locked. As a rule, a transaction must unlock the object after its termination. Therefore, every transaction requires a lock and unlock operation for each data item that is accessed. Such operations are automatically managed and scheduled by the DBMS; the user does not need to be concerned about locking or unlocking data items.

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Lock PRODUCT	
2	T1	Read PROD_QOH	15
3	T1	$\text{PROD_QOH} = 15 + 10$	
4	T1	Write PROD_QOH	25
5	T1	Unlock PRODUCT	
6	T2	Lock PRODUCT	
7	T2	Read PROD_QOH	23
8	T2	$\text{PROD_QOH} = 23 - 10$	
9	T2	Write PROD_QOH	13
10	T2	Unlock PRODUCT	

Shared/Exclusive Locks

The labels “shared” and “exclusive” indicate the nature of the lock. An exclusive lock exists when access is reserved specifically for the transaction that locked the object. The exclusive lock must be used when the potential for conflict exists. A shared lock exists when concurrent transactions are granted read access on the basis of a common lock. A shared lock produces no conflict as long as all the concurrent transactions are read-only.

An exclusive lock is issued when a transaction wants to update (write) a data item and no locks are currently held on that data item by any other transaction. Using the shared/exclusive locking concept, a lock can have three states: unlocked, shared (read), and exclusive (write).

Two transactions conflict only when at least one of them is a Write transaction. Because the two Read transactions can be safely executed at once, shared locks allow several Read transactions to read the same data item concurrently. For example, if transaction T1 has a shared lock on data item X and transaction T2 wants to read data item X, T2 may also obtain a shared lock on data item X.

If transaction T2 updates data item X, an exclusive lock is required by T2 over data item X. *The exclusive lock is granted if and only if no other locks are held on the data item.* Therefore, if a shared or exclusive lock is already held on data item X by transaction T1, an exclusive lock cannot be granted to transaction T2 and T2 must wait to begin until T1 commits. This condition is known as the mutual exclusive rule: only one transaction at a time can own an exclusive lock on the same object.

Although the use of shared locks renders data access more efficient, a shared/exclusive lock schema increases the lock manager’s overhead, for several reasons:

- The type of lock held must be known before a lock can be granted.
- Three lock operations exist: READ_LOCK (to check the type of lock), WRITE_LOCK (to issue the lock), and UNLOCK (to release the lock).
- The schema has been enhanced to allow a lock upgrade (from shared to exclusive) and a lock downgrade (from exclusive to shared).

Although locks prevent serious data inconsistencies, they can lead to two major problems:

- The resulting transaction schedule might not be serializable.

- The schedule might create deadlocks. A deadlock occurs when two transactions wait indefinitely for each other to unlock data. A database deadlock, which is equivalent to traffic gridlock in a big city, is caused when two or more transactions wait for each other to unlock data.

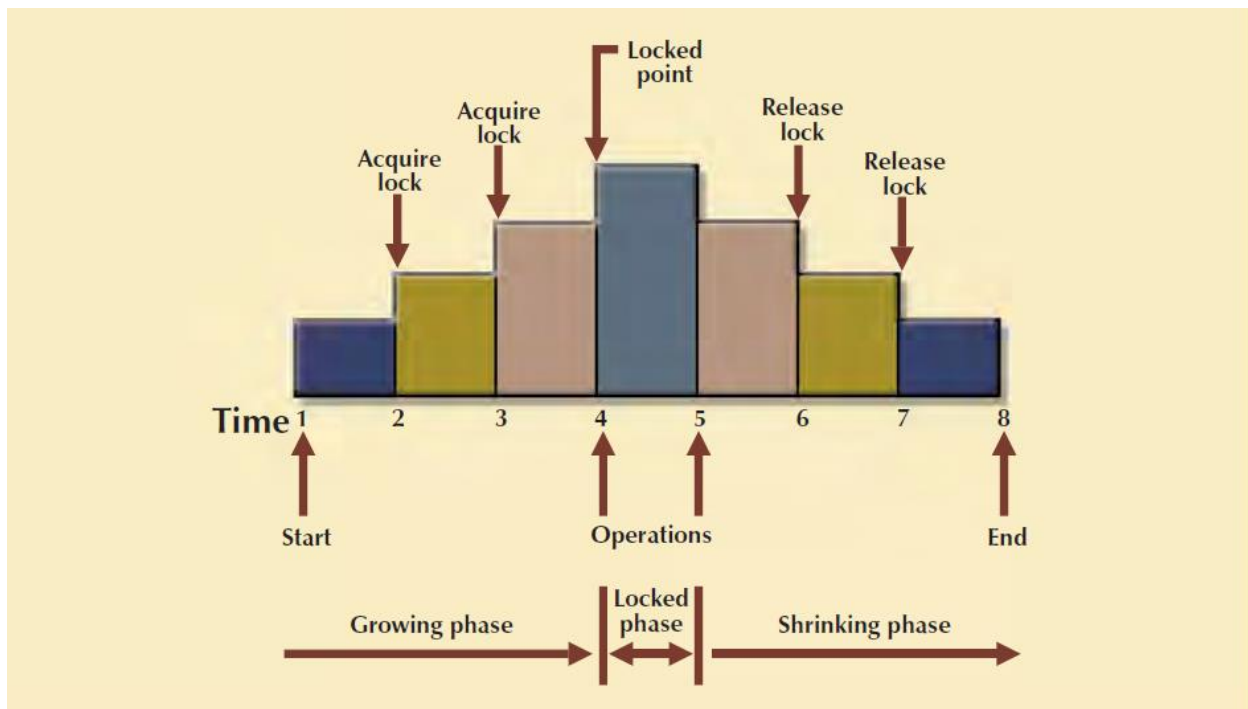
Two-Phase Locking to Ensure Serializability

Two-phase locking defines how transactions acquire and relinquish locks. Two-phase locking guarantees serializability, but it does not prevent deadlocks. The two phases are:

1. A growing phase, in which a transaction acquires all required locks without unlocking any data. Once all locks have been acquired, the transaction is in its locked point.
2. A shrinking phase, in which a transaction releases all locks and cannot obtain any new lock.

The two-phase locking protocol is governed by the following rules:

- Two transactions cannot have conflicting locks.
- No unlock operation can precede a lock operation in the same transaction.
- No data are affected until all locks are obtained—that is, until the transaction is in its locked point.



In this example, the transaction acquires all of the locks it needs until it reaches its locked point. (In this example, the transaction requires two locks.) When the locked point is reached, the data are modified to conform to the transaction's requirements. Finally, the transaction is completed as it releases all of the locks it acquired in the first phase.

Two-phase locking increases the transaction processing cost and might cause additional undesirable effects. One undesirable effect is the possibility of creating deadlocks.

Deadlocks

A deadlock occurs when two transactions wait indefinitely for each other to unlock data. For example, a deadlock occurs when two transactions, T1 and T2, exist in the following mode:

T1 = access data items X and Y

T2 = access data items Y and X

If T1 has not unlocked data item Y, T2 cannot begin; if T2 has not unlocked data item X, T1 cannot continue.

Consequently, T1 and T2 each wait for the other to unlock the required data item. Such a deadlock is also known as a *deadly embrace*.

TIME	TRANSACTION	REPLY	LOCK STATUS	
0			Data X	Data Y
1	T1:LOCK(X)	OK	Unlocked	Unlocked
2	T2: LOCK(Y)	OK	Locked	Unlocked
3	T1:LOCK(Y)	WAIT	Locked	Locked
4	T2:LOCK(X)	WAIT	Locked	Locked
5	T1:LOCK(Y)	WAIT	Locked	Locked
6	T2:LOCK(X)	WAIT	Locked	Locked
7	T1:LOCK(Y)	WAIT	Locked	Locked
8	T2:LOCK(X)	WAIT	Locked	Locked
9	T1:LOCK(Y)	WAIT	Locked	Locked
...				
...				
...				
...				



Deadlocks are possible only when one of the transactions wants to obtain an exclusive lock on a data item; no deadlock condition can exist among *shared* locks.

The three basic techniques to control deadlocks are:

- **Deadlock prevention.** A transaction requesting a new lock is aborted when there is the possibility that a deadlock can occur. If the transaction is aborted, all changes made by this transaction are rolled back and all locks obtained by the transaction are released. The transaction is then rescheduled for execution. Deadlock prevention works because it avoids the conditions that lead to deadlocking.
- **Deadlock detection.** The DBMS periodically tests the database for deadlocks. If a deadlock is found, one of the transactions (the “victim”) is aborted (rolled back and restarted) and the other transaction continues.
- **Deadlock avoidance.** The transaction must obtain all of the locks it needs before it can be executed. This technique avoids the rolling back of conflicting transactions by requiring that locks be obtained in succession.
However, the serial lock assignment required in deadlock avoidance increases action response times.

The choice of the best deadlock control method to use depends on the database environment. For example, if the probability of deadlocks is low, deadlock detection is recommended. However, if the probability of deadlocks is high, deadlock prevention is recommended. If response time is not high on the system’s priority list, deadlock avoidance might be employed.

CONCURRENCY CONTROL WITH TIME STAMPING METHODS

The time stamping approach to scheduling concurrent transactions assigns a global, unique time stamp to each transaction. The time stamp value produces an explicit order in which transactions are submitted to the DBMS. Time stamps must have two properties: uniqueness and monotonicity. Uniqueness ensures that no equal time stamp values can exist, and monotonicity¹ ensures that time stamp values always increase.

All database operations (Read and Write) within the same transaction must have the same time stamp. The DBMS executes conflicting operations in time stamp order, thereby ensuring serializability of the

transactions. If two transactions conflict, one is stopped, rolled back, rescheduled, and assigned a new time stamp value.

The disadvantage of the time stamping approach is that each value stored in the database requires two additional time stamp fields: one for the last time the field was read and one for the last update. Time stamping thus increases memory needs and the database's processing overhead.

Wait/Die and Wound/Wait Schemes

TRANSACTION REQUESTING LOCK	TRANSACTION OWNING LOCK	WAIT/DIE SCHEME	WOUND/WAIT SCHEME
T1 (11548789)	T2 (19562545)	• T1 waits until T2 is completed and T2 releases its locks.	• T1 preempts (rolls back) T2. • T2 is rescheduled using the same time stamp.
T2 (19562545)	T1 (11548789)	• T2 dies (rolls back). • T2 is rescheduled using the same time stamp.	• T2 waits until T1 is completed and T1 releases its locks.

Using the wait/die scheme:

- If the transaction requesting the lock is the older of the two transactions, it will *wait* until the other transaction is completed and the locks are released.
- If the transaction requesting the lock is the younger of the two transactions, it will *die* (roll back) and is rescheduled using the same time stamp.

In short, in the wait/die scheme, the older transaction waits for the younger to complete and release its locks.

In the wound/wait scheme:

- If the transaction requesting the lock is the older of the two transactions, it will preempt (*wound*) the younger transaction (by rolling it back). T1 preempts T2 when T1 rolls back T2. The younger, preempted transaction is rescheduled using the same time stamp.
- If the transaction requesting the lock is the younger of the two transactions, it will wait until the other transaction is completed and the locks are released.

In short, in the wound/wait scheme, the older transaction rolls back the younger transaction and reschedules it.

CONCURRENCY CONTROL WITH OPTIMISTIC METHODS

The optimistic approach is based on the assumption that the majority of the database operations do not conflict. The optimistic approach requires neither locking nor time stamping techniques. Instead, a transaction is executed without restrictions until it is committed. Using an optimistic approach, each transaction moves through two or three phases, referred to as *read*, *validation*, and *write*.

- During the *read phase*, the transaction reads the database, executes the needed computations, and makes the updates to a private copy of the database values. All update operations of the transaction are recorded in a temporary update file, which is not accessed by the remaining transactions.
- During the *validation phase*, the transaction is validated to ensure that the changes made will not affect the integrity and consistency of the database. If the validation test is positive, the transaction goes to the write phase. If the validation test is negative, the transaction is restarted and the changes are discarded.
- During the *write phase*, the changes are permanently applied to the database.
- The optimistic approach is acceptable for most read or query database systems that require few update transactions.

DATABASE RECOVERY MANAGEMENT

Database recovery restores a database from a given state (usually inconsistent) to a previously consistent state. Recovery techniques are based on the atomic transaction property: all portions of the transaction must be treated as a single, logical unit of work in which all operations are applied and completed to produce a consistent database.

If, for some reason, any transaction operation cannot be completed, the transaction must be aborted and any changes to the database must be rolled back (undone). In short, transaction recovery reverses all of the changes that the transaction made to the database before the transaction was aborted.

Critical events can cause a database to become nonoperational and compromise the integrity of the data.

Examples of critical events are:

- *Hardware/software failures.* A failure of this type could be a hard disk media failure, a bad capacitor on a motherboard, or a failing memory bank. Other causes of errors under this category include application program or operating system errors that cause data to be overwritten, deleted, or lost. Some database administrators argue that this is one of the most common sources of database problems.
- *Human-caused incidents.* This type of event can be categorized as unintentional or intentional.
 - An unintentional failure is caused by carelessness by end users. Such errors include deleting the wrong rows from a table, pressing the wrong key on the keyboard, or shutting down the main database server by accident.
 - Intentional events are of a more severe nature and normally indicate that the company data are at serious risk. Under this category are security threats caused by hackers trying to gain unauthorized access to data resources and virus attacks caused by disgruntled employees trying to compromise the database operation and damage the company.
- *Natural disasters.* This category includes fires, earthquakes, floods, and power failures.

Transaction Recovery

Database transaction recovery uses data in the transaction log to recover a database from an inconsistent state to a consistent state.

- The write-ahead-log protocol ensures that transaction logs are always written *before* any database data are actually updated. This protocol ensures that, in case of a failure, the database can later be recovered to a consistent state, using the data in the transaction log.
- Redundant transaction logs (several copies of the transaction log) ensure that a physical disk failure will not impair the DBMS's ability to recover data.
- Database buffers are temporary storage areas in primary memory used to speed up disk operations. To improve processing time, the DBMS software reads the data from the physical disk and stores a copy of it on a "buffer" in primary memory. When a transaction updates data, it actually updates the copy of the data in the buffer because that process is much faster than accessing the physical disk every time. Later on, all buffers that contain updated data are written to a physical disk during a single operation, thereby saving significant processing time.
- Database checkpoints are operations in which the DBMS writes all of its updated buffers to disk. While this is happening, the DBMS does not execute any other requests. A checkpoint operation is also registered in the transaction log. As a result of this operation, the physical database and the transaction log will be in sync. This synchronization is required because update operations update the copy of the data in the buffers and not in the physical database. Checkpoints are automatically scheduled by the DBMS several times per hour.

The database recovery process involves bringing the database to a consistent state after a failure.

Transaction recovery procedures generally make use of deferred-write and write-through techniques.

Deferred-Write Technique

The transaction operations do not immediately update the physical database. Instead, only the transaction log is updated. The database is physically updated only after the transaction reaches its commit point, using information from the transaction log.

If the transaction aborts before it reaches its commit point, no changes (no ROLLBACK or undo) need to be made to the database because the database was never updated.

The recovery process for all started and committed transactions (before the failure) follows these steps:

1. Identify the last checkpoint in the transaction log. This is the last time transaction data was physically saved to disk.
2. For a transaction that started and was committed before the last checkpoint, nothing needs to be done because the data are already saved.
3. For a transaction that performed a commit operation after the last checkpoint, the DBMS uses the transaction log records to redo the transaction and to update the database, using the “after” values in the transaction log. The changes are made in ascending order, from oldest to newest.
4. For any transaction that had a ROLLBACK operation after the last checkpoint or that was left active (with neither a COMMIT nor a ROLLBACK) before the failure occurred, nothing needs to be done because the database was never updated.

Write-through technique/Immediate update

The database is immediately updated by transaction operations during the transaction’s execution, even before the transaction reaches its commit point. If the transaction aborts before it reaches its commit point, a ROLLBACK or undo operation needs to be done to restore the database to a consistent state. In that case, the ROLLBACK operation will use the transaction log “before” values. The recovery process follows these steps:

1. Identify the last checkpoint in the transaction log. This is the last time transaction data were physically saved to disk.
2. For a transaction that started and was committed before the last checkpoint, nothing needs to be done because the data are already saved.
3. For a transaction that was committed after the last checkpoint, the DBMS redoes the transaction, using the “after” values of the transaction log. Changes are applied in ascending order, from oldest to newest.
4. For any transaction that had a ROLLBACK operation after the last checkpoint or that was left active (with neither a COMMIT nor a ROLLBACK) before the failure occurred, the DBMS uses the transaction log records to ROLLBACK or undo the operations, using the “before” values in the transaction log. Changes are applied in reverse order, from newest to oldest.

DATA WAREHOUSING (Introduction)

A data warehouse is a subject-oriented, integrated, time-variant, non-updateable collection of data used in support of management decision-making processes and business intelligence (Inmon and Hackathorn, 1994). The meaning of each of the key terms in this definition follows:

- *Subject-oriented* A data warehouse is organized around the key subjects (or high-level entities) of the enterprise. Major subjects may include customers, patients, students, products, and time.
- *Integrated* The data housed in the data warehouse are defined using consistent naming conventions, formats, encoding structures, and related characteristics gathered from several internal systems of record and also often from sources external to the organization. This means that the data warehouse holds the one version of “the truth.”
- *Time-variant* Data in the data warehouse contain a time dimension so that they may be used to study trends and changes.
- *Non-updateable* Data in the data warehouse are loaded and refreshed from operational systems, but cannot be updated by end users.

Data warehousing is the process whereby organizations create and maintain data warehouses and extract meaning and inform decision making from their informational assets through these data warehouses. Successful data warehousing requires following proven data warehousing practices, sound project management, strong organizational commitment, as well as making the right technology decisions.

Data warehousing emerged as a result of advances in the field of information systems over several decades. The following were some key advances:

- Improvements in database technology, particularly the development of the relational data model and relational database management systems (RDBMSs)
- Advances in computer hardware, particularly the emergence of affordable mass storage and parallel computer architectures
- The emergence of end-user computing, facilitated by powerful, intuitive computer interfaces and tools
- Advances in middleware products that enable enterprise database connectivity across heterogeneous platforms (Hackathorn, 1993)

In general, certain trends in organizations encourage the need for data warehousing; these trends include the following:

- *No single system of record* Almost no organization has only one database. Seems odd, doesn't it? Because of the heterogeneous needs for data in different operational settings, because of corporate mergers and acquisitions, and due to the sheer size of many organizations, multiple operational databases exist.
- *Multiple systems are not synchronized* It is difficult, if not impossible, to make separate databases consistent. Even if the metadata are controlled and made the same by one data administrator, the data values for the same attributes will not agree. This is because of different update cycles and separate places where the same data are captured for each system. Thus, to get one view of the organization, the data from the separate systems must be periodically consolidated and synchronized into one additional database.
- *Organizations want to analyze the activities in a balanced way* many organizations have implemented some form of a balanced scorecard—metrics that show organization results in financial, human, customer satisfaction, product quality, and other terms simultaneously. To ensure that this multidimensional view of the organization shows consistent results, a data warehouse is necessary.
- *Customer relationship management* Organizations in all sectors are realizing that there is value in having a total picture of their interactions with customers across all touch points. Different touch points (e.g., for a bank, these touch points include ATMs, online banking, tellers, electronic funds transfers, investment portfolio management, and loans) are supported by separate operational systems.
- *Supplier relationship management* managing the supply chain has become a critical element in reducing costs and raising product quality for many organizations. Organizations want to create strategic supplier partnerships based on a total picture of their activities with suppliers, from billing, to meeting delivery dates, to quality control, to pricing, to support. Data about these different activities can be locked inside separate operational systems (e.g., accounts payable, shipping and receiving, production scheduling, and maintenance). ERP systems have improved this situation by bringing many of these data into one database.

OVERVIEW: DISTRIBUTED DATABASES

A distributed database is a single logical database that is spread physically across computers in multiple locations that are connected by a data communications network.

The distributed database is still centrally administered as a corporate resource while providing local flexibility and customization. It requires multiple instances of a database management system (or DBMSs) running at each remote site.

Objectives and Trade-offs

A major objective of distributed databases is to provide ease of access to data for users at many different locations. To meet this objective, the distributed database system must provide:

- Location transparency, which means that a user (or user program) using data for querying or updating need not know the location of the data. Any request to retrieve or update data from any site is automatically forwarded by the system to the site or sites related to the processing request. Ideally, the user is unaware of the distribution of data, and all data in the network appear as a single logical database stored at one site. In this ideal case, a single query can join data from tables in multiple sites as if the data were all in one site.
- Local autonomy, which is the capability to administer a local database and to operate independently when connections to other nodes have failed (Date, 2003). With local autonomy, each site has the capability to control local data, administer security, log transactions, recover when local failures occur, and provide full access to local data to local users when any central or coordinating site cannot operate. In this case, data are locally owned and managed, even though they are accessible from remote sites.

A distributed DBMS provides location transparency (defined earlier), replication transparency, failure transparency, and concurrency transparency.

Replication transparency: A design goal for a distributed database, which says that although a given data item may be replicated at several nodes in a network, a programmer or user may treat the data item as if it were a single item at a single node. Also called fragmentation transparency.

Failure transparency: A design goal for a distributed database, which guarantees that either all the actions of each transaction are committed or else none of them is committed.

Concurrency transparency: A design goal for a distributed database, with the property that although a distributed system runs many transactions, it appears that a given transaction is the only activity in the system. Thus, when several transactions are processed concurrently, the results must be the same as if each transaction were processed in serial order.

THE DATABASE LIFECYCLE

